

Preentrega Módulo 2

Arquitectura Multi-Agente para Soporte Técnico

Cecilia Manzano

1. Descripción del proyecto

Se desarrolló en n8n un sistema multi-agente para la gestión automática de tickets de soporte técnico.

La arquitectura implementa el patrón **Manager–Worker**, separando la tarea de clasificación de los tickets de su resolución técnica. El objetivo es evitar un agente monolítico y dividir las responsabilidades entre especialistas con alcances claramente definidos.

El sistema está compuesto por:

- Un **Agente Manager**, encargado de analizar y clasificar cada ticket.
 - Un **Worker especializado en Soporte a Usuarios**.
 - Un **Worker especializado en Infraestructura**.
 - Una tercera opción hacia un **supervisor humano** cuando el ticket no puede clasificarse con suficiente claridad.
 - Una vía de escape en caso de que el agente no se limite a alguna de las tres opciones de clasificación.
-

2. Arquitectura general

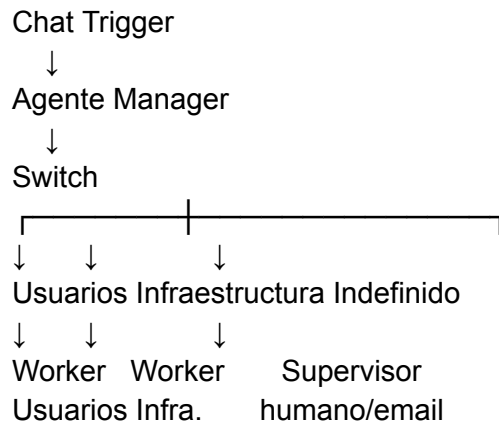
El flujo principal recibe el ticket mediante el chat de n8n.

El Agente Manager analiza el lenguaje natural del usuario y devuelve una clasificación estructurada dentro de una taxonomía cerrada:

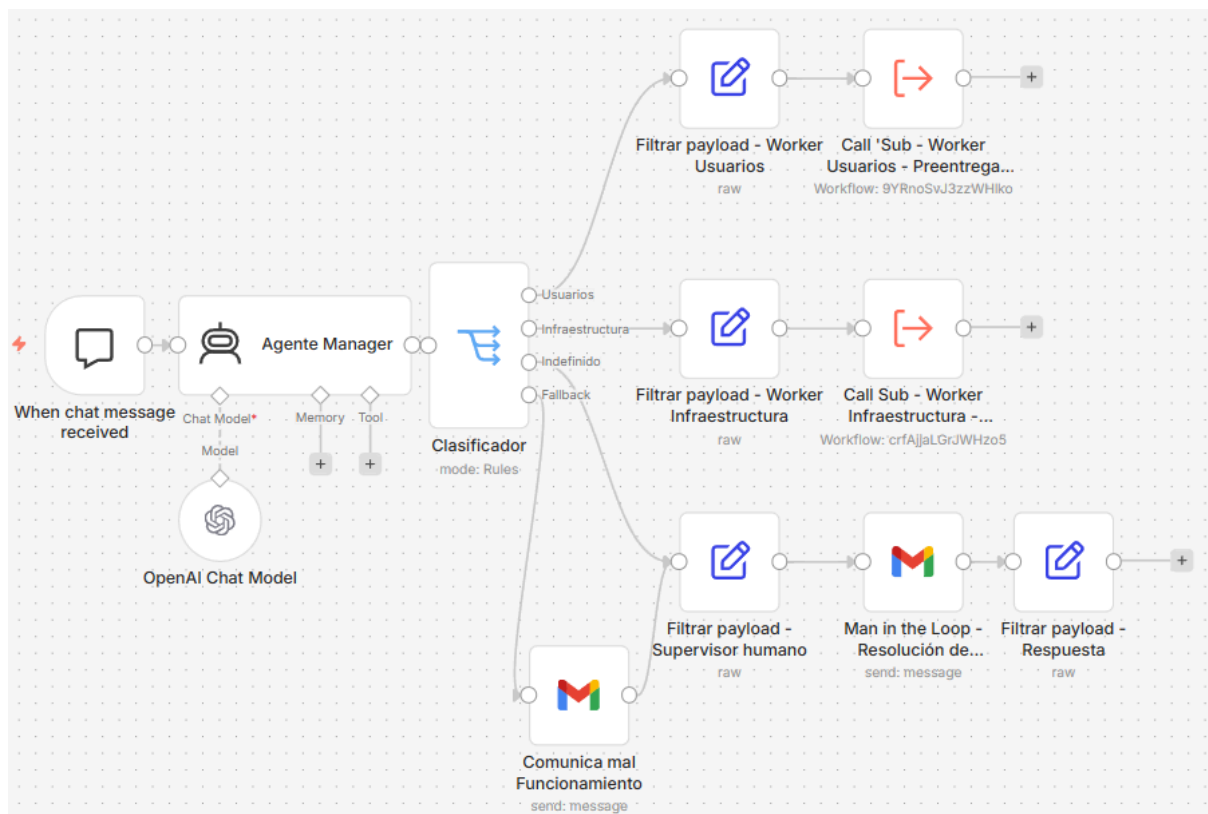
```
{"tarea":"usuarios"}  
{"tarea":"infraestructura"}  
{"tarea":"indefinido"}
```

Un nodo **Switch** utiliza este resultado para enrutar el ticket hacia el especialista correspondiente.

La arquitectura general es:



Captura — Workflow principal



3. Agente Manager y enrutamiento

El Manager funciona exclusivamente como **agente de triaje**.

No intenta resolver los problemas técnicos. Su responsabilidad consiste en comprender el ticket y determinar qué especialista debe intervenir.

La taxonomía usada es:

Usuarios: problemas asociados principalmente a dispositivos finales, computadoras, notebooks, celulares, sistemas operativos, aplicaciones, periféricos y soporte técnico directo al usuario.

Infraestructura: problemas relacionados con redes LAN, Ethernet, switches, routers, access points, cableado, Wi-Fi corporativo, servidores y demás componentes de infraestructura tecnológica.

Indefinido: tickets ambiguos o con información insuficiente para asignarlos responsablemente a uno de los dos especialistas. La clasificación “indefinido” actúa como vía en los casos que no es claro el límite de si se trata de un asunto de Soporte Técnico de Usuarios o de Infraestructura. Estos casos son derivados mediante Gmail a una persona de soporte técnico para su revisión manual.

Como vía de escape en casos excepcionales en que el Agente Manager no devuelva alguna de las 3 opciones estándar, previo a la intervención de una persona de Soporte técnico para casos indefinidos, se le remitirá el caso a los Técnicos de automatización.

Lógica de enrutamiento

El Workflow, mediante el Agente Manager no solo contempla la posibilidad de que se derive el flujo hacia un Worker u otro, sino que contempla los casos donde la “frontera” entre una temática u otra (Usuarios/Infraestructura) no sea muy clara y eso lo resuelve con una tercer opción que es “Indefinido”. El manager debe, en función al prompt System, devolver como respuesta un Json con alguna de las 3 opciones: “usuario”, “infraestructura” o “indefinido”.

El flujo podrá derivar los caminos a partir de un nodo Switch llamado Clasificador, en el cual están embebidas las reglas de. si el mensaje del Agente es = “usuario”, irá por el primer camino que es hacia el agente especialista en Soporte Técnico relativo a Usuarios, si es = “infraestructura” irá por el segundo camino, que es hacia el agente especialista en Soporte Técnico relativo a Infraestructura y si es = “indefinido” irá por el tercero que dirige hacia man in the loop, donde una persona de Soporte Técnico lo revisará y abordará. Para el caso en que ocurriera un error con el agente, por ejemplo no hizo caso al formato de salida que estaba definido, el Switch guiará el flujo por el cuarto camino que es el fall back, que lo guiará hacia la comunicación del fallo a un técnico de automatización, pero así mismo proseguirá el flujo como si se tratara de un “indefinido”, para que el ticket igualmente pueda tener una resolución, ya que el mensaje del ticket no habrá sufrido inconvenientes.

A continuación se muestran capturas de pantalla de:

1. la definición del formato json que debe devolver el Agente Manager
2. El nodo Switch en contexto
3. Las reglas del nodo Switch
4. Emails en la bandeja de entrada, en caso de camino “indefinido” y en caso de error en el Agente Manager

Edit System Message

System Message

6. FORMATO DE SALIDA

Respondé SIEMPRE exclusivamente con un objeto JSON válido.

Para soporte a usuarios:

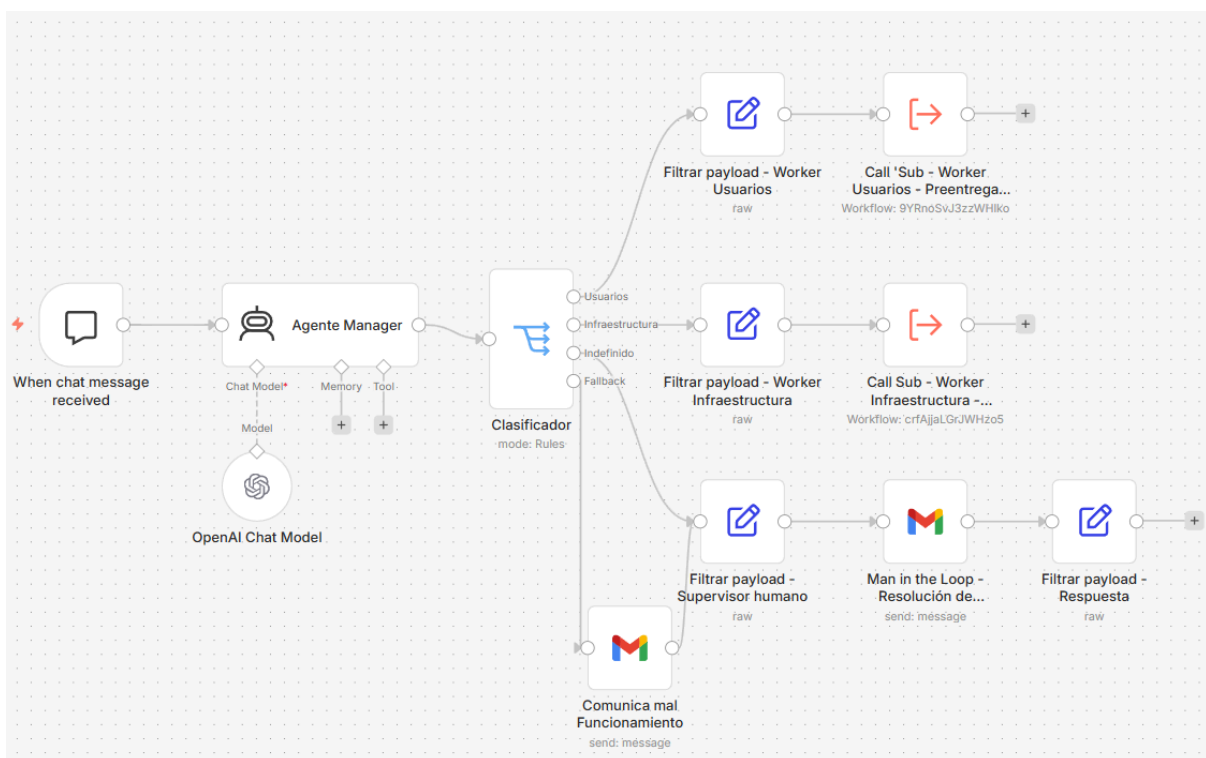
```
{ "tarea": "usuarios" }
```

Para soporte de infraestructura:

```
{ "tarea": "infraestructura" }
```

Cuando no pueda determinarse correctamente:

```
{ "tarea": "indefinido" }
```



Parameters

Settings

Execute step

Mode

Rules

Routing Rules

+

Routing Rule 1

fx

{{ \$json.output }}

T contains

usuarios

Rename Output

Output Name

Usuarios

Routing Rule 2

fx

{{ \$json.output }}

T contains

infraestructura

Rename Output

Output Name

Infraestructura

Routing Rule 3

fx

{{ \$json.output }}

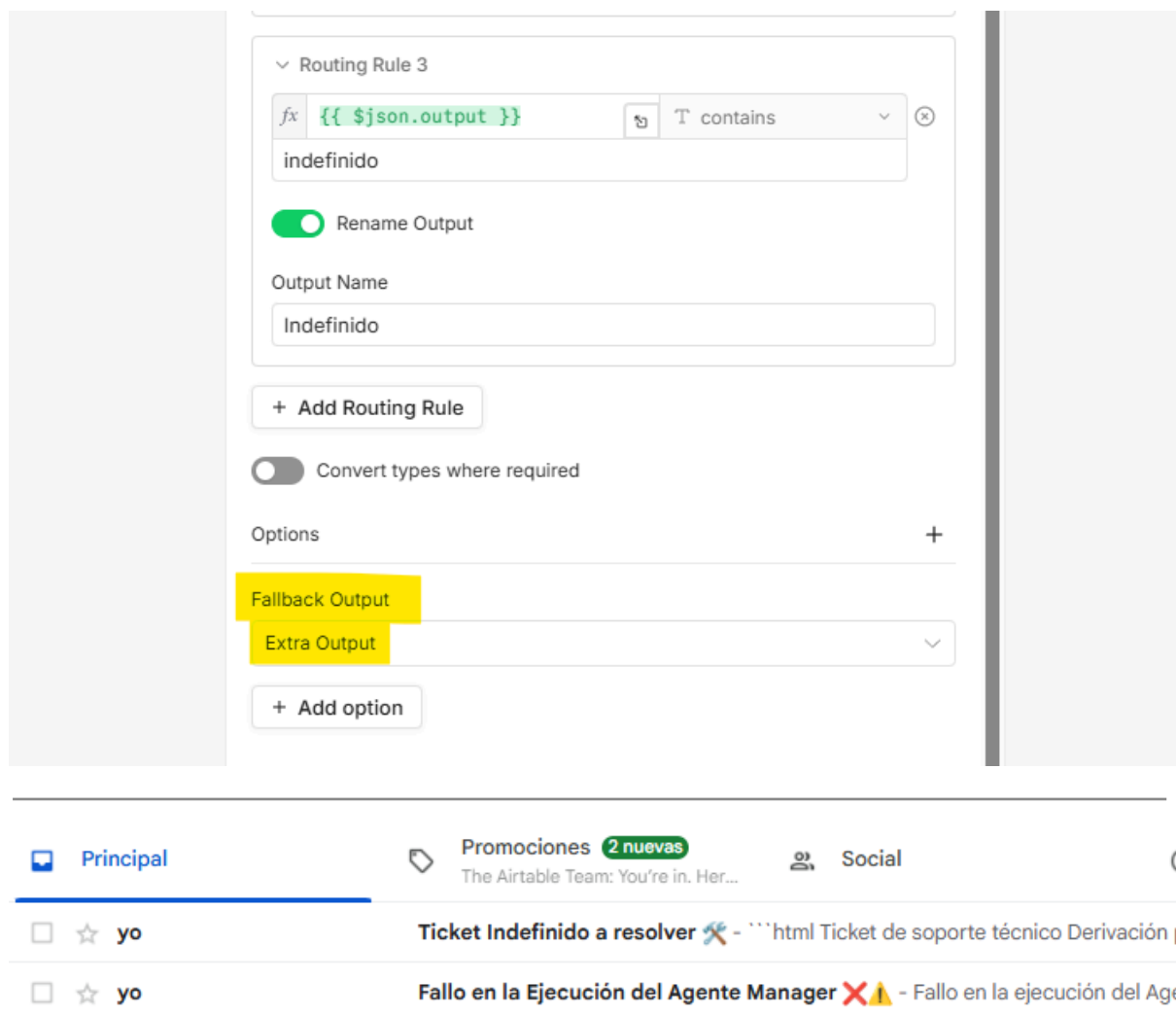
T contains

indefinido

Rename Output

Output Name

Indefinido



Nodos de Set Node o Edit Data

Con los nodos de Set field lo que se logra es limpiar la información que ingresará en el sub workflow de los workers, de esta forma minimiza errores quitando el ruido ocasionado por la llegada de información que luego no será necesaria y hace que el flujo y latencia sean más fluidas. Se implementó un nodo set previo a cada uno de los subflujos workers.

Set previo al subflujo Worker Usuarios

Previo al set, el workflow arrastra información de 3 nodos más las variables propias que posee por defecto el Workflow :

- Clasificador (Switch)
- Agente Manager (AI Agent)
- Wham chat message received (Chat)
- Variables and context

A continuación se visualiza la información de entrada al nodo Set Node o Edit Data:

Filtrar payload - Worker Usuarios

INPUT

Q

SchemaTableJSON

▼  Clasificador1 item

T

output

{"tarea":"usuarios"}

▼  Agente Manager1 item

T

output

{"tarea":"usuarios"}

▼  When chat message received ⚡1 item

T

action

sendMessage

T

sessionId

6baa7dc7e75442dd8ab98af578f50284

T

chatInput

hola necesito ayuda con mi windows, está desactualizado

▼ Variables and context

T

\$now

2026-08-20T18:54:19.126-04:00

T

\$today

2026-08-20T00:00:00.000-04:00



\$vars



▼



 \$execution

T

id

[filled at execution time]

T

mode

test

T

resumeUrl

The URL for resuming a 'Wait' node

▼



 \$workflow

T

id

sUP56aXmCxb7Pcs9

T

name

☒

active

false

Mediante el nodo Set se realiza un saneamiento de la información, permitiendo que únicamente pase al sub workflow aquella información que va a necesitar el sub workflow, que es el ticket que levantó la persona y que requiere ser analizado y resuelto por el Worker Usuario.

A continuación se visualiza en el nodo Set Node o Edit Data, el Json con la información definida que recibirá el Sub Workflow, que oficia de filtro y el Output resultante.

Parameters

Settings

Execute step

Mode

JSON

JSON

fx

```
{
  "ticket": "{{ $('When chat message received').item.json.chatInput }}"
}
```

```
{  "ticket": "hola necesito ayuda con mi windows, está desactualizad..."
```

Include Other Input Fields

Options

+

+ Add option

OUTPUT

Schema

1 item

```
[
  {
    "ticket": "hola necesito ayuda con mi windows, está desactualizado"
  }
]
```

Set previo al subflujo Worker Infraestructura

Así como en el caso del Sub Workflow Usuarios, previo al de Infraestructura, se aplicó el mismo criterio para el saneamiento de la información a ingresar a dicho Sub Workflow.

A continuación se visualiza la información de entrada al nodo Set Node o Edit Data:

 Filtrar payload - Worker Infraestructura

INPUT

SchemaTableJSON

Clasificador1 item

Toutput

{"tarea":"infraestructura"}

Agente Manager1 item

Toutput

{"tarea":"infraestructura"}

When chat message received ⚡1 item

Taction

sendMessage

TsessionId

6baa7dc7e75442dd8ab98af578f50284

TchatInput

Va a ser necesario cablear la red para que los endpoint tengan calidad asegurada de internet

Variables and context

T\$now

2026-08-20T19:17:36.627-04:00

T\$today

2026-08-20T00:00:00.000-04:00

\$vars

\$execution

Tid

[filled at execution time]

Tmode

test

TresumeUrl

The URL for resuming a 'Wait' node

\$workflow

Tid

sUP56aXmCxb7Pcs9

Tname

active

false

A continuación la información que pasará al Sub Workflow Infraestructura, que es el ticket de requerimiento de soporte técnico a nivel de Infraestructura, y el Output resultante post filtro:

Parameters

Settings

Execute step

Mode

JSON

JSON

fx

```
{
  "ticket": "{{ $('When chat message received').item.json.chatInput }}"
}
```

```
{
  "ticket": "Va a ser necesario cablear la red para que los endpoin..."
}
```

Include Other Input Fields

Options

+

+ Add option

OUTPUT

Schema

Table

JSON

1 item

```
[
  {
    "ticket": "Va a ser necesario cablear la red para que los endpoint tengan calidad asegurada de internet"
  }
]
```

4. Worker de Soporte a Usuarios

El Worker de Usuarios fue desarrollado como un **sub-workflow independiente**.

Su alcance incluye principalmente:

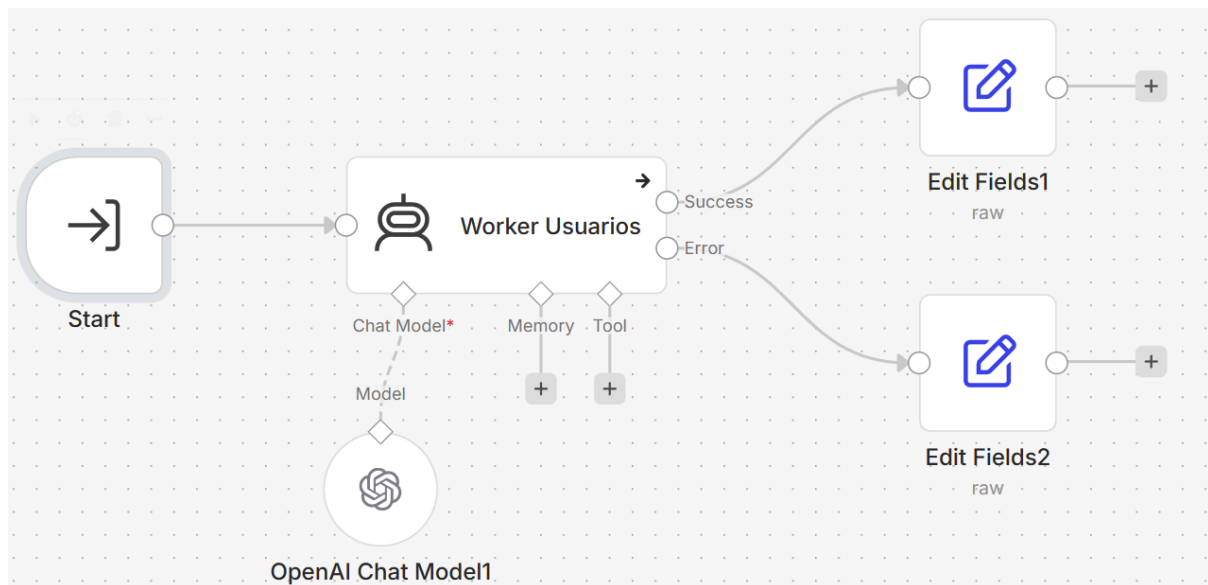
- Computadoras y notebooks.
- Windows y macOS.
- Hardware de dispositivos finales.
- Software y aplicaciones.
- Drivers.
- Periféricos.
- Celulares y tablets.
- Problemas locales de conectividad cuando la falla está limitada al dispositivo del usuario.

El sub-workflow comienza mediante **When Executed by Another Workflow**, permitiendo que sea invocado desde el Manager.

Recibe como entrada únicamente el dato necesario para realizar su tarea:

```
{  
  "ticket": "descripción del problema"  
}
```

Captura — Sub-workflow Worker Usuarios



5. Worker de Infraestructura

El segundo Worker también funciona como un sub-workflow independiente y está especializado en infraestructura IT.

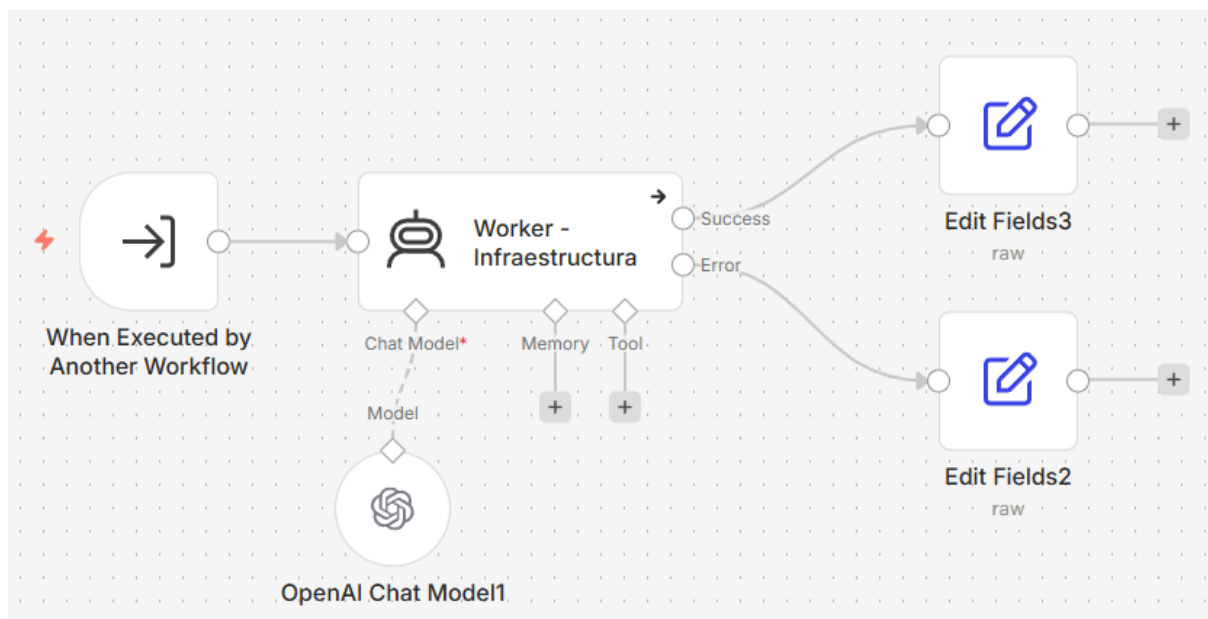
Su alcance comprende:

- Redes LAN y Ethernet.
- Switches.
- Routers.
- Access Points.
- Wi-Fi corporativo.
- VLAN y direccionamiento.
- Cableado estructurado.
- Servidores físicos y virtuales.
- Sistemas operativos de servidores.
- Servicios centrales e infraestructura compartida.

Al igual que el Worker de Usuarios, recibe un contrato de entrada reducido al dato necesario:

```
{
  "ticket": "descripción del problema"
}
```

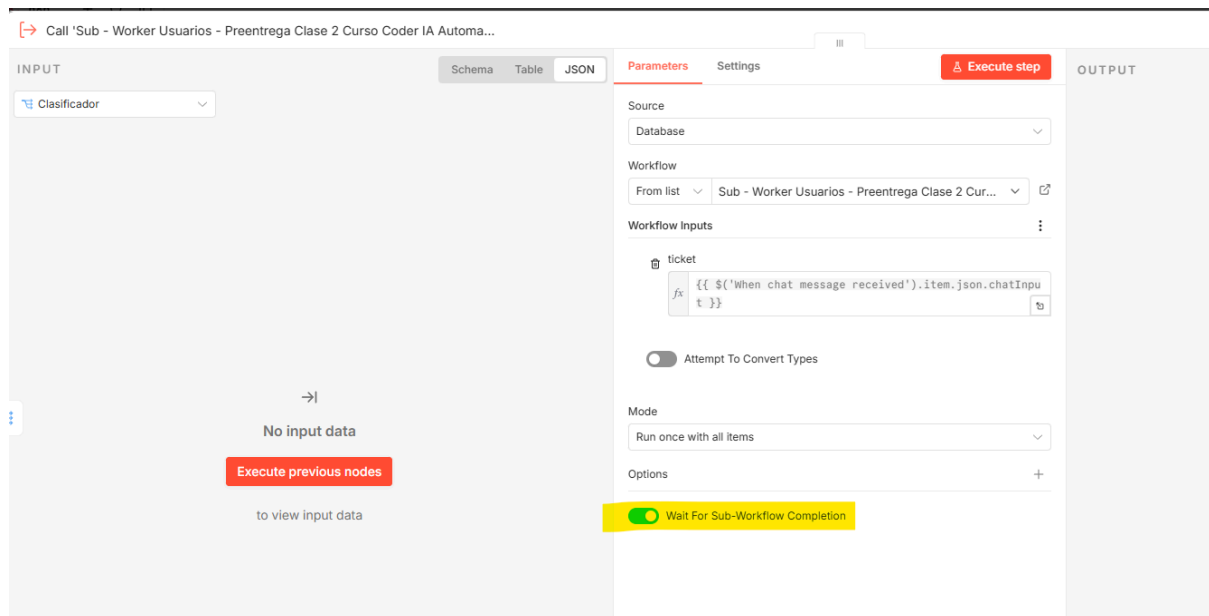
Captura — Sub-workflow Worker Infraestructura



6. Delegación y sincronización

Desde el workflow principal se utilizan nodos **Execute Sub-workflow / Call Sub-workflow** para invocar al especialista correspondiente.

La ejecución está configurada para **esperar la finalización del Worker**, de modo que el flujo principal no continúe hasta que el especialista termine de procesar el ticket y devuelva su resultado. A continuación se visualiza la configuración activada:



Esto permite mantener una ejecución sincronizada:

Manager



Delegación al Worker



Espera de finalización



Respuesta estructurada del Worker



Continuación del flujo principal

7. Contrato de salida de los Workers

Los Workers devuelven una estructura JSON predecible, evitando transmitir información innecesaria al workflow principal.

Ejemplo de ejecución correcta:

```
{
  "estado del agente": "exitoso",
  "worker": "infraestructura",
  "ticket": "El servidor no responde",
  "respuesta": "Respuesta técnica generada por el especialista"
}
```

Para el Worker de Usuarios se utiliza la misma estructura, modificando el especialista:

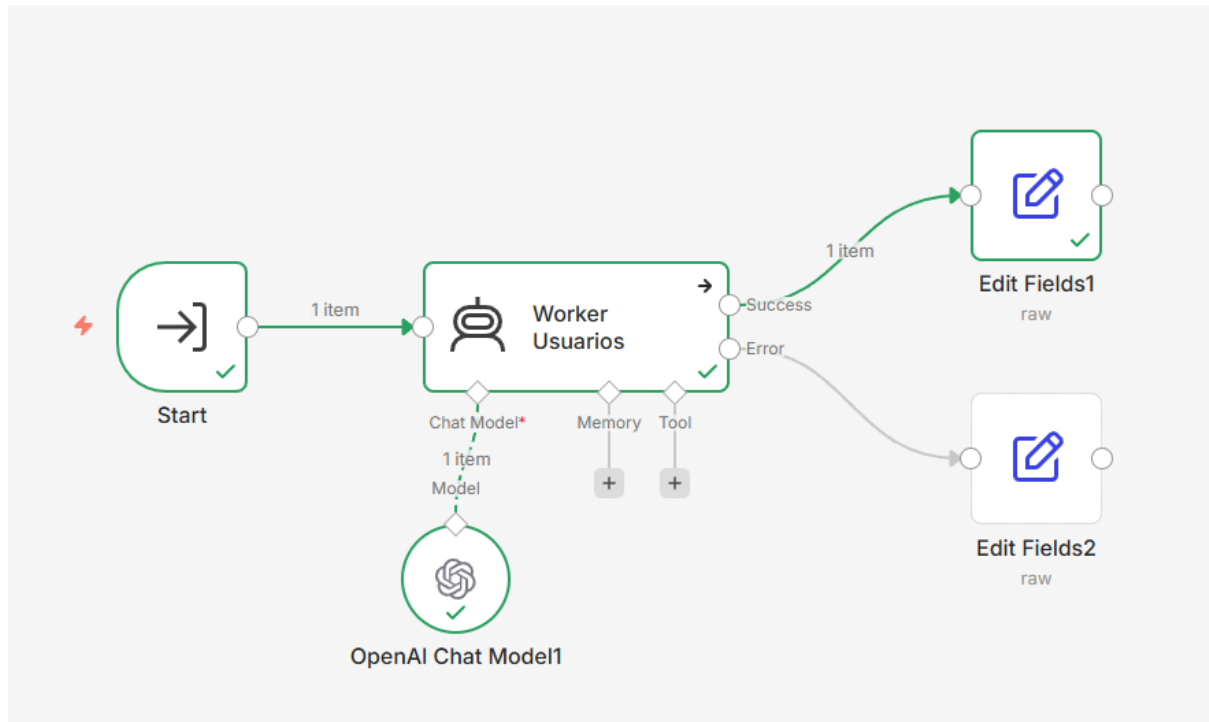
```
{
  "estado del agente": "success",
```

```

"worker": "usuarios",
"ticket": "La notebook no inicia",
"respuesta": "Respuesta técnica generada por el especialista"
}

```

Ejemplo de contrato de salida caso exitoso Worker Usuario:



Expression

Anything inside `{{ }}` is JavaScript. [Learn more](#)

```

{
  "estado del agente": "exitoso",
  "worker": "usuarios",
  "ticket": {{ JSON.stringify($('Start').item.json.ticket) }},
  "respuesta": {{ JSON.stringify($json.output) }}
}

```

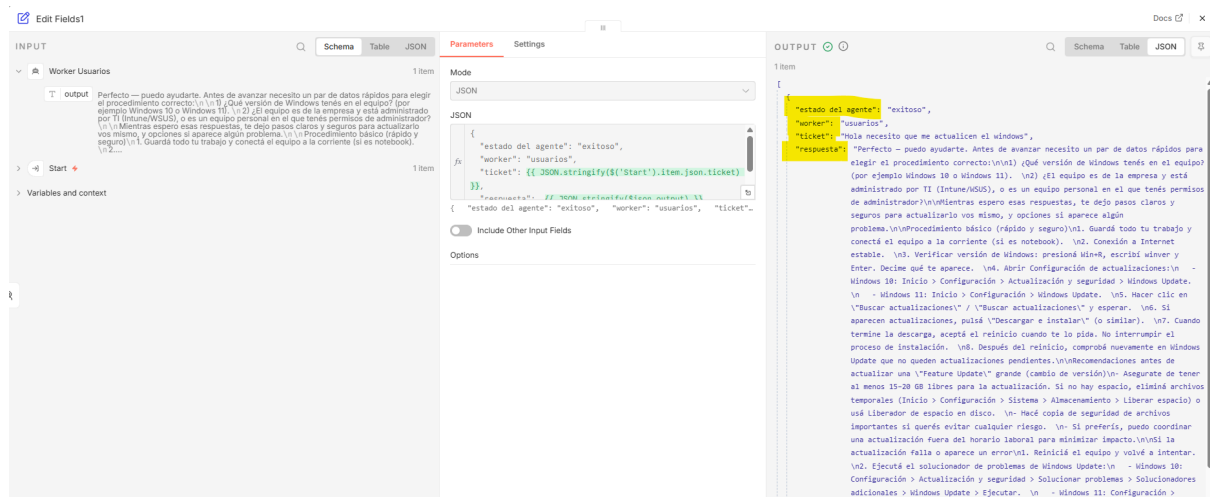
Result

Item

```

{
  "estado del agente": "exitoso",
  "worker": "usuarios",
  "ticket": [Execute previous nodes for preview],
  "respuesta": [Execute previous nodes for preview]
}

```



También se contempla una salida estructurada ante errores, permitiendo mantener un contrato de datos predecible independientemente del resultado de la ejecución.

```
{
  "estado del agente": "error",
  "worker": "infraestructura",
  "ticket": "El servidor no responde",
  "respuesta": "No fue posible procesar el ticket."
}
```

Para el Worker de Usuarios se utiliza la misma estructura, modificando el especialista:

```
{
  "estado del agente": "error",
  "worker": "usuarios",
  "ticket": "La notebook no inicia",
  "respuesta": "No fue posible procesar el ticket."
}
```

8. Modularidad y separación de responsabilidades

La principal mejora respecto de una arquitectura monolítica es la separación explícita de responsabilidades.

El **Manager decide**, pero no resuelve.

El **Worker de Usuarios** resuelve únicamente problemas correspondientes a dispositivos y usuarios finales.

El **Worker de Infraestructura** se ocupa exclusivamente de redes, servidores e infraestructura.

Los casos que no pueden clasificarse de forma responsable son derivados a una persona.

Esta estructura permite que cada especialista pueda ser probado, mantenido o modificado independientemente sin alterar el resto del sistema.

9. Repositorio GitHub

Los workflows exportados en formato JSON se encuentran en el siguiente repositorio:

Repositorio:

<https://github.com/CeciliaIM/Cecilia-Manzano---AI-Automation-Avanzado-Curso-Coderhouse--->

Workflow principal – Manager:

<https://github.com/CeciliaIM/Cecilia-Manzano---AI-Automation-Avanzado-Curso-Coderhouse---/blob/main/Preentrega%20Clase%202%20Curso%20Coder%20IA%20Automation%20avanzado%20-%20Arquitectura%20multiagente%20-%20Soporte%20t%C3%A9cnico.json>

Sub-workflow – Worker Usuarios:

<https://github.com/CeciliaIM/Cecilia-Manzano---AI-Automation-Avanzado-Curso-Coderhouse---/blob/main/Sub%20-%20Worker%20Usuarios%20-%20Preentrega%20Clase%202%20Curso%20Coder%20IA%20Automation%20avanzado%20-%20Arquitectura%20multiagente.json>

Sub-workflow – Worker Infraestructura:

<https://github.com/CeciliaIM/Cecilia-Manzano---AI-Automation-Avanzado-Curso-Coderhouse---/blob/main/Sub%20-%20Worker%20Infraestructura%20-%20Preentrega%20Clase%202%20Curso%20Coder%20IA%20Automation%20avanzado%20-%20Arquitectura%20multiagente.json>

10. Conclusión

La solución transforma el proceso de soporte técnico en una arquitectura multi-agente modular basada en el patrón **Manager-Worker**.

El Manager centraliza la decisión y el enrutamiento, mientras que los Workers ejecutan tareas especializadas mediante sub-workflows independientes y contratos JSON definidos. La ruta “indefinido” incorpora supervisión humana para aquellos casos donde la automatización no dispone de información suficiente para delegar el ticket de forma segura a un especialista específico.

El resultado es una arquitectura más clara, mantenible y escalable que un único agente encargado de resolver todos los tipos de incidencias.